

■ JavaScript Complete Notes (Simple + Full Explanation)

■ 1. Global Scope

■ Definition:

Global scope means a variable is accessible from anywhere in your code (inside or outside functions/blocks).

■ Simple Example:

```
var a = 10;
```

```
function test() {  
  console.log(a); // ■ can access  
}
```

```
console.log(a); // ■ can access
```

■ Here `a` is global → usable everywhere

■ Global Scope with var, let, const

1 ■ var (Old way)

```
var x = 5;
```

```
function demo() {  
  console.log(x); // ■ works  
}
```

✓ If declared outside → global

■ Can be redeclared and changed easily

■ Global → YES

■ Safe → NO

2 ■ let (Better way)

```
let y = 20;
```

```
function demo() {  
  console.log(y); // ■ works  
}
```

✓ If declared outside → global

✓ Cannot redeclare in same scope

✓ Safer than var

■ Global → YES

■ Safe → YES

3 ■ const (Best way)

```
const z = 50;
```

```
function demo() {  
  console.log(z); // ■ works  
}
```

- ✓ If declared outside → global
- ✓ Cannot change value
- ✓ Most safe

■ Global → YES
■ Change → ■ No

■■ Important Note

```
{  
  let a = 10;  
}  
console.log(a); // ■ Error
```

■ Because let & const are block scoped

■ Final Table

Keyword	Global	Change	Safe
-----	-----	-----	-----
var	■ Yes	■ Yes	■ No
let	■ Yes	■ Yes	■ Yes
const	■ Yes	■ No	■ Yes

■ Trick:

■ Global = declared outside = usable everywhere

■ 2. Event Loop (Full Understanding)

■ Parts:

- ■ Call Stack → where code runs
- ■ Web APIs → background helpers
- ■ Microtask Queue → Promises
- ■ Macrotask Queue → setTimeout
- ■ Event Loop → manager

■ Example:

```
console.log("Start");  
setTimeout(() => {
```

```
    console.log("Timeout");  
  }, 0);
```

```
Promise.resolve().then(() => {  
  console.log("Promise");  
});
```

```
console.log("End");
```

■ Output:

Start

End

Promise

Timeout

■ Flow:

1. Run sync code → Start
2. setTimeout → Web API → Macrotask queue
3. Promise → Microtask queue
4. Run End
5. Event loop starts
6. Run Microtask → Promise
7. Run Macrotask → Timeout

■ Rule:

■ Sync → Microtask → Macrotask

■ 3. Synchronous Code

■ Runs line by line immediately

Example:

```
console.log("A");  
console.log("B");
```

Output:

A

B

■ 4. Callback

■ Function that runs later

Example:

```
setTimeout(() => {  
  console.log("Hello");  
});
```

```
}, 2000);
```

■ Runs after delay

■ 5. Promise

■ Modern async method

Example:

```
Promise.resolve().then(() => {  
  console.log("Promise");  
});
```

■ Runs before setTimeout

■ 6. Web APIs / Node APIs

■ These are helpers outside JavaScript

They handle:

- ■ setTimeout
- ■ fetch()
- ■ events
- ■ file system

■ Flow:

JS → Web API → Queue → Event Loop → Run

■ Real Example:

```
setTimeout(() => {  
  console.log("Hi");  
}, 2000);
```

■ JS sends it to Web API

■ After 2 sec → goes to queue

■ Event loop runs it

■ 7. Task Queues

■ What is Queue?

■ Waiting line for async tasks

■ Microtask Queue (High Priority)

■ VIP line ■

Includes:

- Promises (.then)
- async/await
- queueMicrotask

■ Macrotask Queue (Low Priority)

■ Normal line ■

Includes:

- setTimeout
- setInterval
- DOM events
- setImmediate

■ Example:

```
console.log("Start");
```

```
setTimeout(() => console.log("Timeout"), 0);
```

```
Promise.resolve().then(() => console.log("Promise"));
```

```
console.log("End");
```

■ Output:

Start

End

Promise

Timeout

■ Rule:

■ Microtask > Macrotask

■ FINAL SUMMARY

■ JavaScript works like this:

1. Run all sync code
2. Run all promises (microtask)
3. Run setTimeout (macrotask)

■ SUPER FINAL TRICK

■ Sync → Promise → Timeout

■ Real Life Understanding

■ You are doing work

- Current work = Sync
- Urgent work = Promise
- Normal work = setTimeout

■ You finish:

1. Current work
2. Urgent work
3. Normal work